
Automated
Sudoku
Solver
A MATLAB Project Uses Image Processing
Tools to Extract Puzzle & solves.

Course No: EEE 312

Course Teacher:

1. Dr. Maruf Ahmed

   Assistant Professor, EEE, BUET

2. Abdullah Jubair Bin Iqbal
   Lecturer, EEE, BUET

Submitted by:

1. Moshiur Rahman- 2006099
2. Shahriar Rashid Khan Sifat-2006117
3. Tanvir Arafat Fahim-2006118
4. Md. Sazidur Rahman- 2006121

Introduction:

Sudoku puzzles, with their simple rules and challenging arrangements, have become a beloved pastime for puzzle enthusiasts worldwide. These puzzles, typically presented in a 9x9 grid, require logical deduction and problem-solving skills to fill in the grid's empty cells with numbers from 1 to 9, ensuring that each row, column, and 3x3 subgrid contains every digit exactly once. While Sudoku solving traditionally involves pen and paper or digital interfaces, the emergence of automated solvers using image processing techniques presents an exciting frontier in puzzle-solving technology.

The objective of this project is to develop an automated Sudoku solver leveraging the power of image processing algorithms implemented in MATLAB. Our solver aims to streamline the puzzle-solving process by accepting an image of a Sudoku puzzle as input and producing a solved version of the puzzle as output. By harnessing image processing techniques, we can extract the puzzle grid, identify the digits within each cell, and employ advanced Sudoku-solving strategies to efficiently complete the puzzle.

In this report, we will delve into the intricacies of our automated Sudoku solver, elucidating the methodology behind each step of the process. We will discuss the image processing techniques employed for puzzle extraction, including grid detection and cell segmentation. Additionally, we will explore the methods used for digit recognition within the puzzle cells, ranging from optical character recognition (OCR) to template matching algorithms.

Furthermore, we will provide detailed insights into the implementation of our solver in MATLAB, elucidating the specific functions, algorithms, and techniques utilized. Through thorough testing and evaluation, we will analyze the performance of our solver, considering factors such as accuracy, speed, and robustness across various Sudoku puzzle types and complexities.

In summary, this project represents a convergence of traditional puzzle-solving methodologies with cutting-edge technology, demonstrating the potential of image processing techniques to revolutionize Sudoku solving. Through this report, we invite readers to embark on a journey into the realm of automated puzzle-solving and to envision the myriad possibilities that lie ahead.

Background of Image Processing:

Aspects of Image Processing

- Image Enhancement: Processing an image so that the result is more suitable for a particular application. (sharpening or de-blurring an out of focus image, highlighting edges,improving image contrast, or brightening an image, removing noise)
- Image Restoration: This may be considered as reversing the damage done to an image by a known cause. (removing of blur caused by linear motion, removal of optical distortions)
- Image Segmentation: This involves subdividing an image into constituent parts, or isolating certain aspects of an image. (finding lines, circles, or particular shapes in an image, in an aerial photograph, identifying cars, trees, buildings, or roads.

Photo and Digital Images:

A digital image differs from a photo in that the values are all discrete as Digital processor can deal with only Digital Images. Photo can be shown as 2D continuous function f(x,y).where f is the intensity and x, y are spatial coordinate. To convert a photo to Digital Image it should be sampled and quantized. Every sampled value is like a dot. So, a digital image can be considered as a large array of discrete dots, each of which has a brightness associated with it. These dots are called picture elements, or more simply pixels. The pixels surrounding a given pixel constitute its neighborhood A neighborhood can be characterized by its shape in the same way as a matrix.

Digital Image representation:

Result of Sampling and Quantization is matrix of real number. Let, f(x,y) is sampled in such a way that it has M row and N column. So, size of image is M*N.

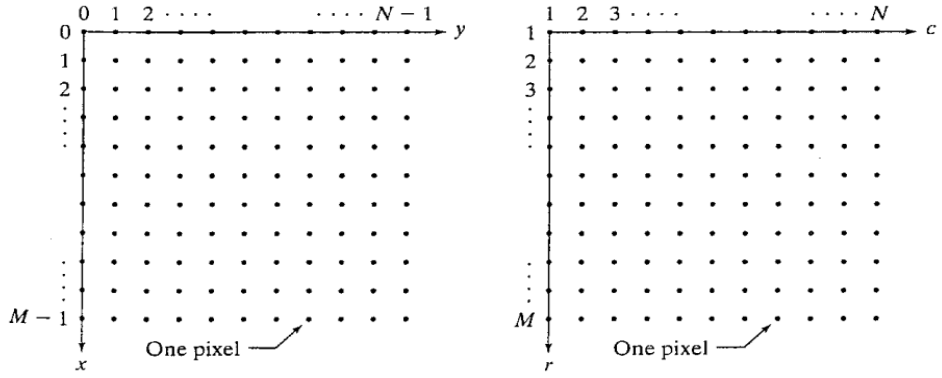


Image as matrix:

The Coordinate system in above and preceding Discussion lead to the following representation of Digital Image:

$$f(x,y) = \begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & & \vdots \\ f(M-1,0) & f(M-1,1) & \cdots & f(M-1,N-1) \end{bmatrix}$$

The right side of this equation is a digital image by definition. Each element of this array is called an *image element*, *picture element*, *pixel*, or *pel*. The terms *image* and *pixel* are used throughout the rest of our discussions to denote a digital image and its elements.

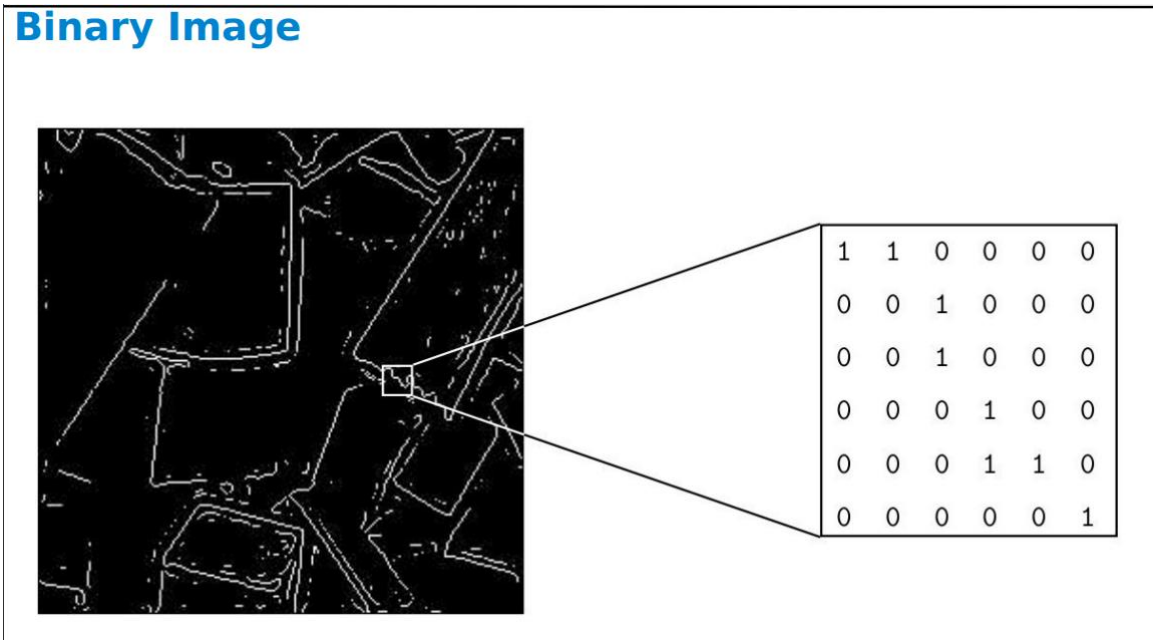
A digital image can be represented naturally as a MATLAB matrix:

$$\mathtt{f} = \begin{bmatrix} \mathtt{f(1,1)} & \mathtt{f(1,2)} & \cdots & \mathtt{f(1,N)} \\ \mathtt{f(2,1)} & \mathtt{f(2,2)} & \cdots & \mathtt{f(2,N)} \\ \vdots & \vdots & & \vdots \\ \mathtt{f(M,1)} & \mathtt{f(M,2)} & \cdots & \mathtt{f(M,N)} \end{bmatrix}$$

Book Reference: Digital Image Processing Using MATLAB. Author: Rafael C. Gonzalez.

Types of Digital Images:

• **Binary**: Each pixel is just black or white. Since there are only two possible values for each pixel (0,1), we only need one bit per pixel.



• Grayscale: Each pixel is a shade of gray, normally from 0 (black) to 255 (white). This range means that each pixel can be represented by eight bits, or exactly one byte. Other greyscale ranges are used, but generally they are a power of 2.

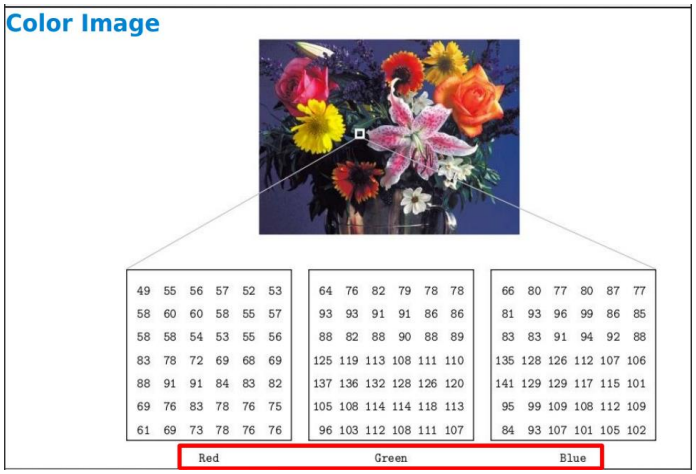


Color Image:

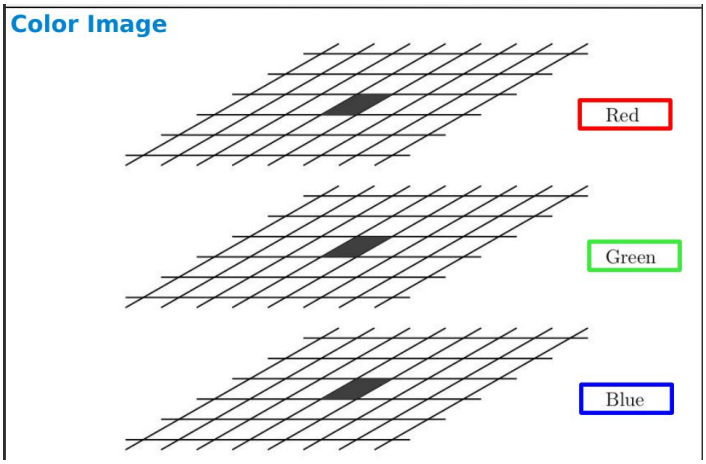
A color model is a method for specifying colors in some standard way. It generally consists of a 3D coordinate system and a subspace of that system in which each color is represented by a single point.

• **True Color, or RGB**: Each pixel has a particular color; that color is described by the amount of red, green and blue in it. If each of these components has a range 0–255, this gives a total of

256^(3) different possible colors. Such an image is a “stack” of three matrices; representing the red, green and blue values for each pixel. This means that for every pixel there correspond 3 values.



In this model, each color is represented as 3 values R, G and B, indicating the amounts of red, green and blue which make up the color.



Thresholding:

Single thresholding: A grayscale image is turned into a binary image by first choosing a gray level T in the original image, and then turning every pixel black or white according to whether its gray value is greater than or less than T.

- A pixel becomes white if its gray level is > T
- A pixel becomes black if its gray level is <= T

Double thresholding: Here we choose two values T1 and T2 and apply a thresholding operation as:

- A pixel becomes white if its gray level between T1 and T2
- A pixel becomes black if its gray level is otherwise



In this project, MATLAB extract text from Digital Binary Image.

Image References: Digital Image Processing Using Matlab.

Author: Haris Papasaika-Hanusch .

Institute of Geodesy and Photogrammetry, ETH Zurich

Connected Component:

1. 4-Connected Components:

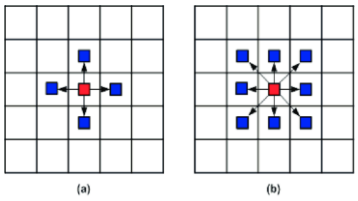
In a 4-connected image, a pixel is considered connected to its four immediate neighbors: above, below, left, and right. This means that if you start from any pixel in the image and traverse to its neighbors using only horizontal and vertical movements, you can reach any other pixel in the same connected component. In simpler terms, two pixels are considered part of the same component if they share an edge (but not necessarily a corner).

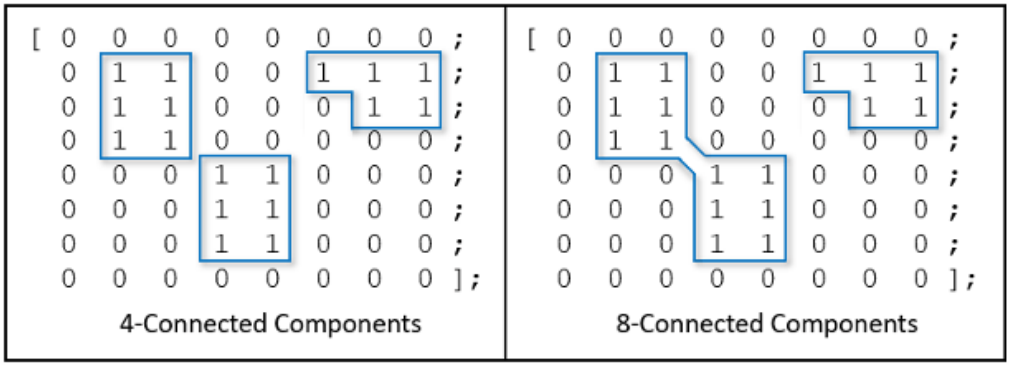
This connectivity definition is commonly used when the diagonal connections between pixels are not relevant or need to be ignored in the analysis. For example, in tasks like object counting or labeling connected regions in a binary image, the 4-connectedness assumption simplifies the computation and interpretation of results.

8-Connected Components:

In contrast, an 8-connected image considers a pixel connected to its eight neighboring pixels: the four immediate neighbors (above, below, left, and right) as well as the four diagonal neighbors. This means that two pixels are part of the same connected component if they share either an edge or a corner.

The 8-connectedness assumption is more inclusive than the 4-connectedness assumption and considers diagonal connections between pixels. This connectivity definition is often used when diagonal connections are meaningful in the context of the image analysis task. For example, in applications where object boundaries are not strictly aligned with the image axes, such as edge detection or contour tracing, the 8-connectedness assumption provides a more accurate representation of object connectivity.





Necessity of Connected Component:

Connected component analysis (CCA) is an important technique in computer vision that is used for a wide range of applications, such as image segmentation, object recognition, and motion tracking. The basic idea behind CCA is to group pixels in an image that are connected and form a single object or entity.

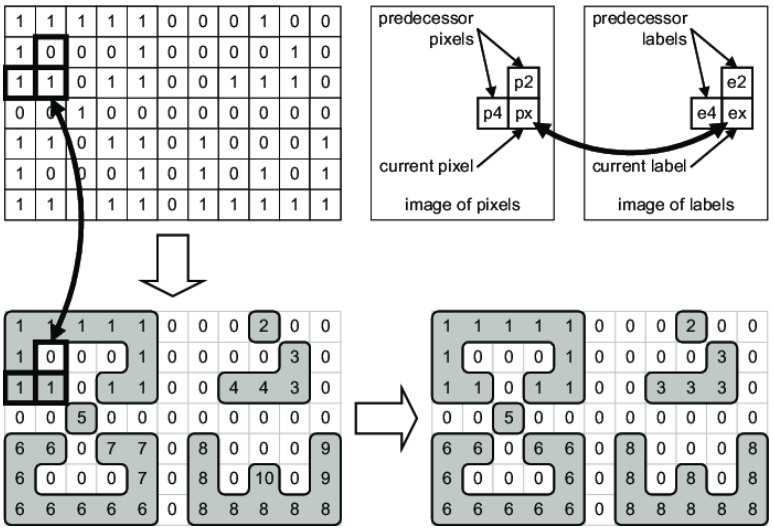
Here are some reasons why connected component analysis is important in computer vision:

1. **Image segmentation:**
   One of the primary applications of CCA is in image segmentation, where it is used to group pixels that belong to the same object or region of interest. This is useful in a variety of computer vision tasks, such as object detection, tracking, and recognition.

2. **Object recognition:**
   CCA can also be used to identify and classify objects in an image or video stream. By grouping pixels that belong to the same object, it becomes easier to extract features and recognize objects based on their shape, color, or texture

Ref: https://www.scaler.com/topics/connected-component-analysis-in-image-processing

Leveling of 4- Connected Components:

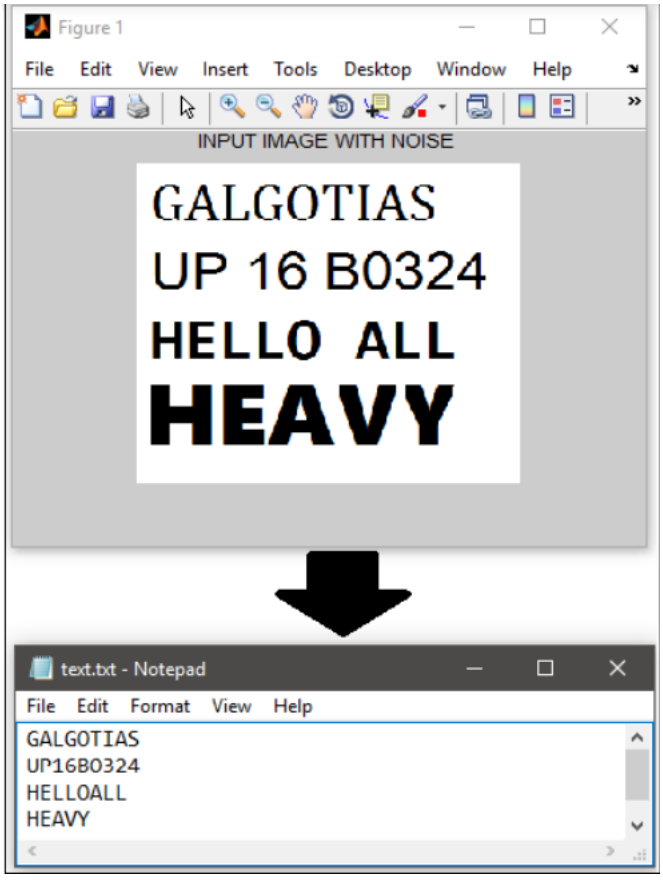


OCR:

Optical Character Recognition (OCR) is a technology designed to automatically identify text within images, enabling users to extract and manipulate text content through an optical process.

In human perception, our eyes serve as the optical mechanism, transmitting visual information to the brain for interpretation. However, individual abilities to understand this input can vary due to various factors.

OCR technology mimics the human ability to recognize and comprehend text, aiding in tasks such as editing and searching for words within scanned documents, including paper documents, PDF files, and images. While OCR is inspired by human reading capabilities, it does not fully replicate the complexity and nuances of human comprehension.



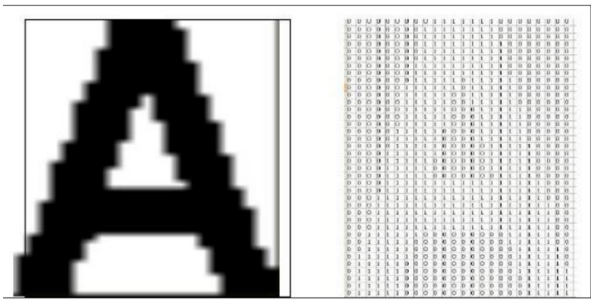


Fig: Character Extraction in form of Matrix

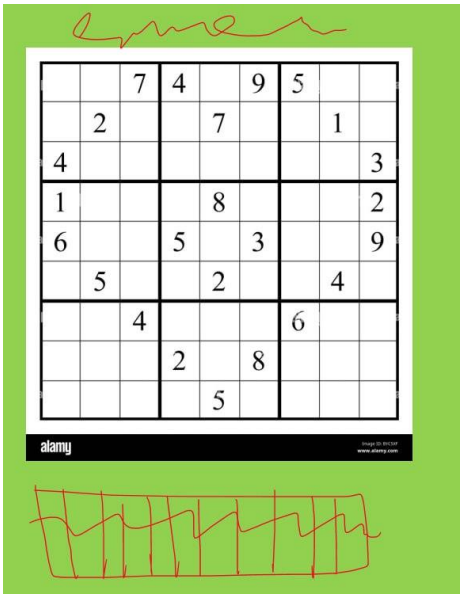
Methodology:

1. Resetting all variables, closing any open figures, and clearing the command window.
2. Loading and displaying an image named "random.jpg".
3. Converting the image to grayscale and showing the grayscale version.
4. Applying adaptive thresholding to binarize the grayscale image and displaying the result.
5. Inverting the binarized image.
6. Identifying and labeling the largest connected component.
7. Determining the coordinates of the labeled region.
8. Cropping the region of interest based on the coordinates.
9. Initializing variables for the sudoku matrix and cell dimensions.
10. Iterating through each cell in the sudoku grid:
    a. Extracting individual cells from the cropped image.
    b. Performing Optical Character Recognition (OCR) on each cell to recognize digits.
    c. Converting recognized text to numeric values and updating the sudoku matrix accordingly.

Converting the solved Matrix to a RGB image

Implementation the Algorithm in MATLAB:

About Image:



```
clear variables ;close all ;clc ;
sudoku=imread('2.jpg'); %Loading Main Image
sz=size(sudoku);
whos sudoku %Name, size, Bytes, Class
info=imfinfo('2.jpg')
```

Size of Image:

```
sz =

  745   577     3

Name         Size                Bytes  Class    Attributes

sudoku     745x577x3           1289595  uint8
```

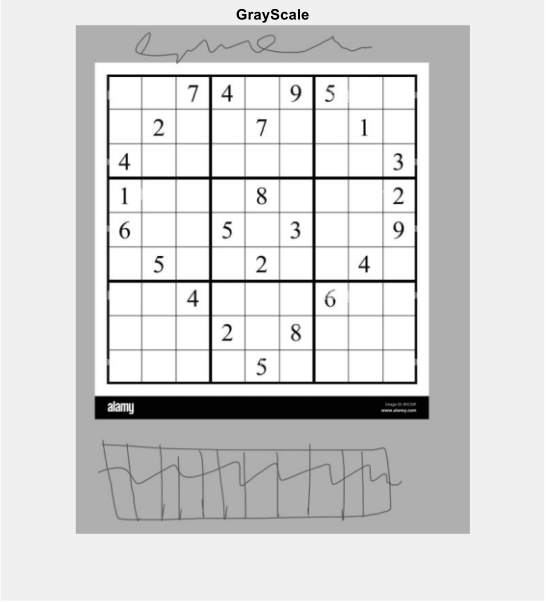
```
info =

  struct with fields:

          Filename: 'C:\Users\Moshiur Rahman\Desktop\DSP project\sudoku function\2.jpg'
      FileModDate: '28-Feb-2024 12:41:34'
         FileSize: 47988
           Format: 'jpg'
    FormatVersion: ''
            Width: 577
           Height: 745
         BitDepth: 24
        ColorType: 'truecolor'
  FormatSignature: ''
  NumberOfSamples: 3
     CodingMethod: 'Huffman'
    CodingProcess: 'Progressive'
          Comment: {}
```

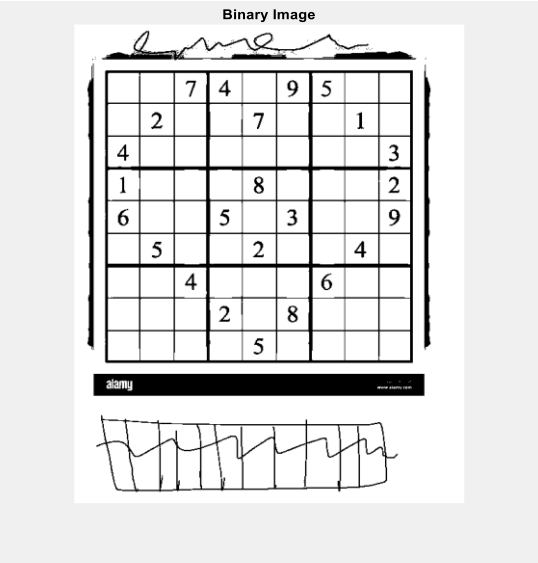
Converting to Grayscale:

```
%rgb to gray
Grayscale=rgb2gray(sudoku);
imshow(Grayscale) ; title('GrayScale') ;
```



Converting to Binary Image:

```
%binarizing image
bw = imbinarize(Grayscale,'adaptive','ForegroundPolarity','dark','Sensitivity',0.5);
figure(3)
imshow(bw) ; title('Binary Image')
```



Laveling the Largest Connected Componect:

```
%extracting largest connected component
IM=imcomplement(bw)  %0 gets 1 and 1 gets zero
bw2 = bwareafilt(IM,1,'largest'); %Extract Objects from Binary Image by size

%1 specifies Number of objects to keep
% largest  means>= criteria of filtering objects

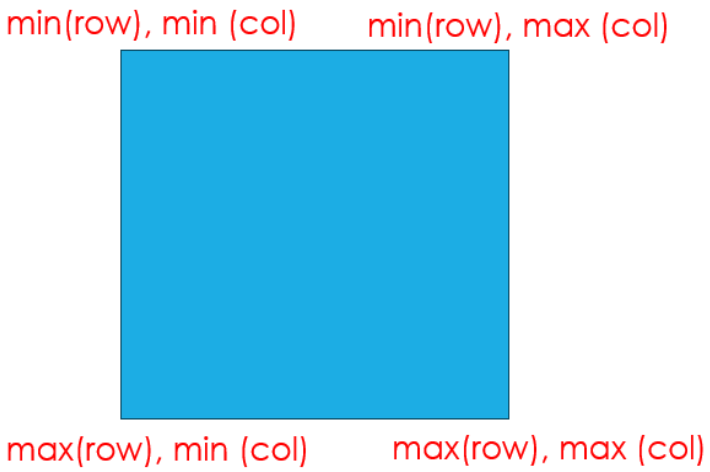

%labeling largest connected component
label=bwlabel(bw2); %connected component= set of adjecent Pixels in Binary image
```

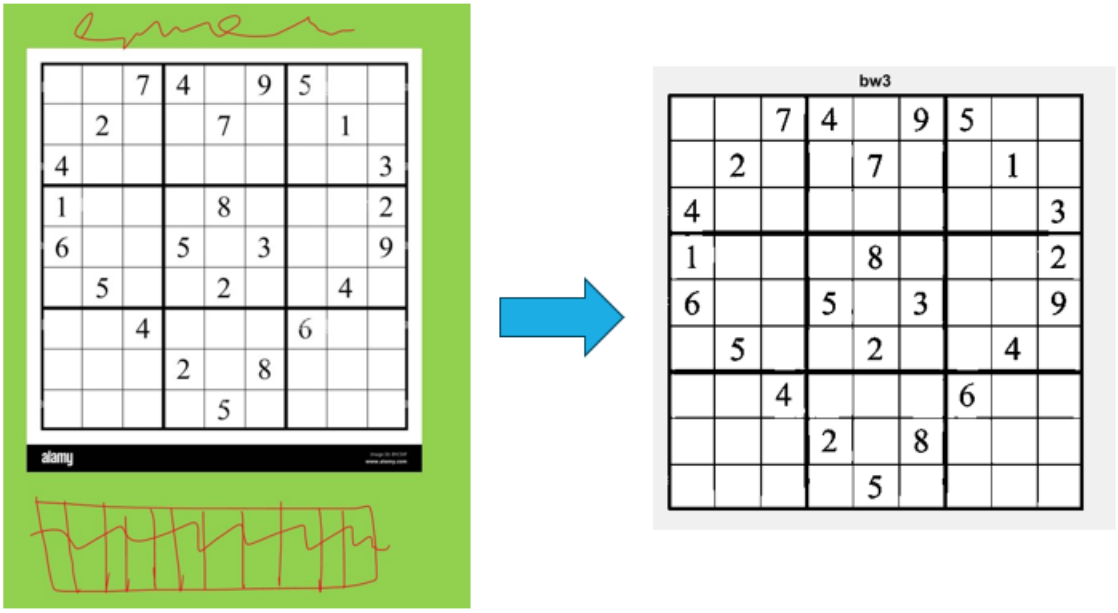
After labeling the maximum connected areas, we bring out the co-ordinates of the pixels for which the pixel value is 1.

[row,col]=find(label==1);

Then, with the help of maximum and minimum value of the co-ordinates we cropped the region of interest or the sudoku grid from the binary image ‘bw’.

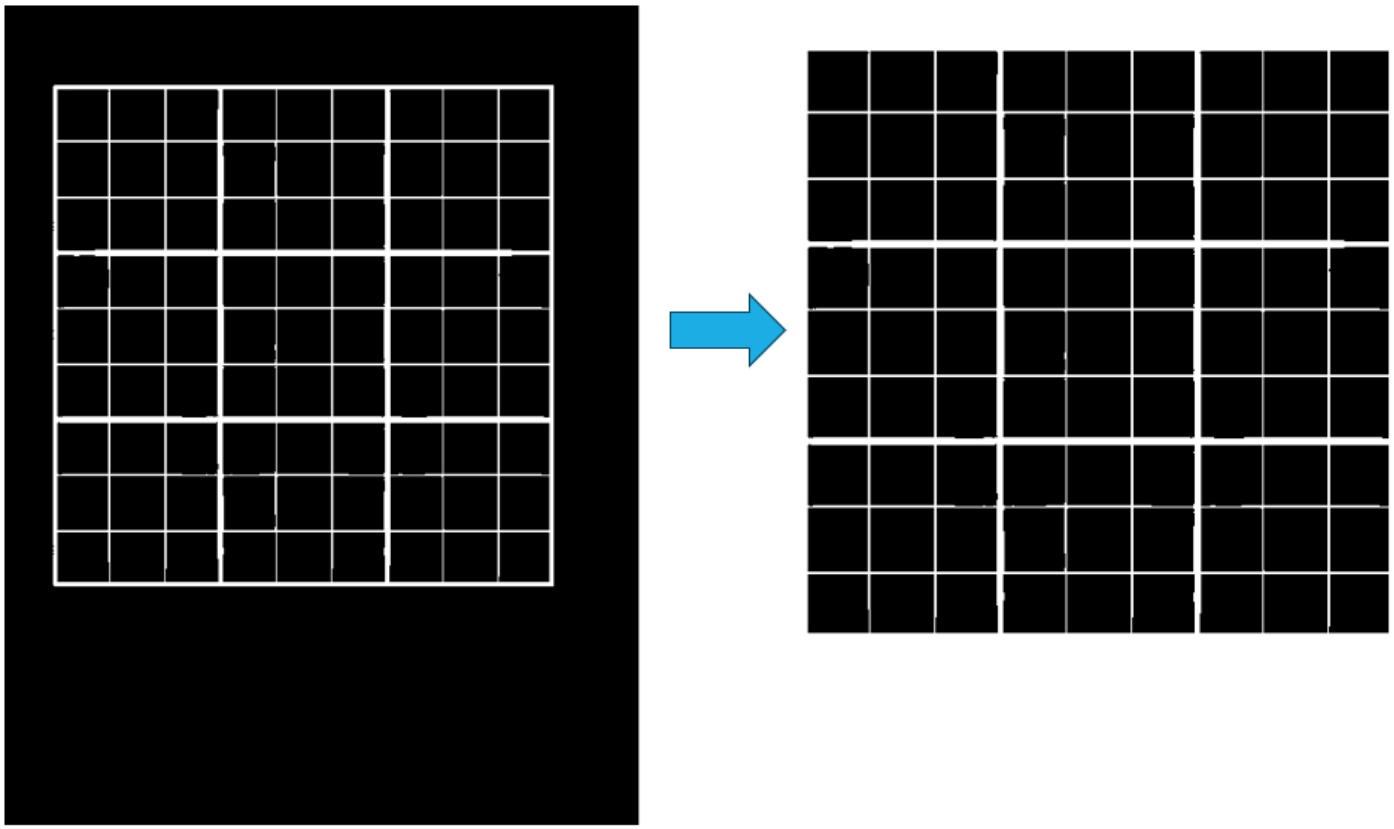
```
%image cropping
bw3=bw(min(row):max(row),min(col):max(col));
```





In ‘bw2’ variable we have stored the maximum connected area and the other objects were removed from that image. We also cropped the maximum connected area part from the image ‘bw2’.

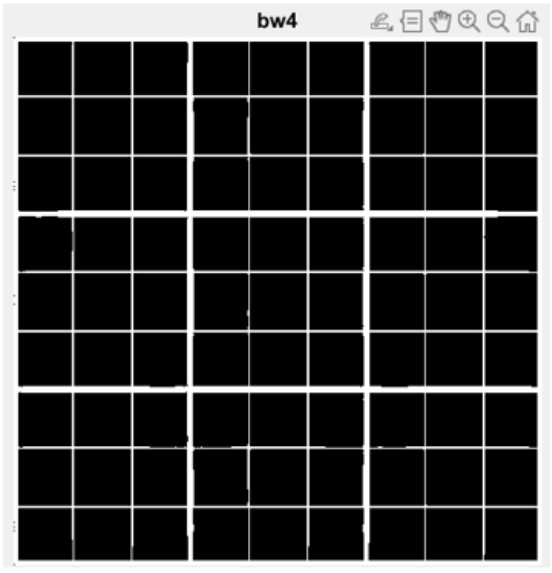
bw4=bw2(min(row):max(row),min(col):max(col));

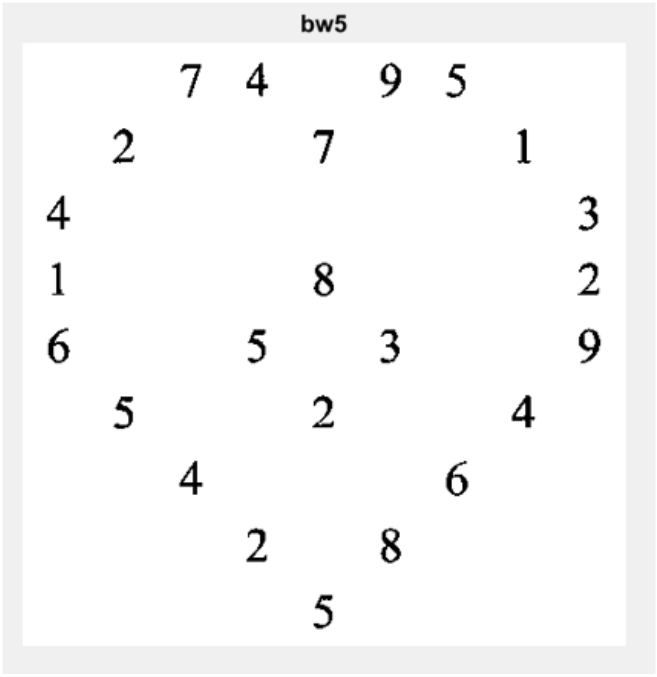


After that, we have done OR operation between two cropped image ‘bw4’ and ‘bw3’ to find the image ‘bw5’ containing only the digits or in this way we have removed all the gridlines from the image.

bw5=bw4|bw3;

;





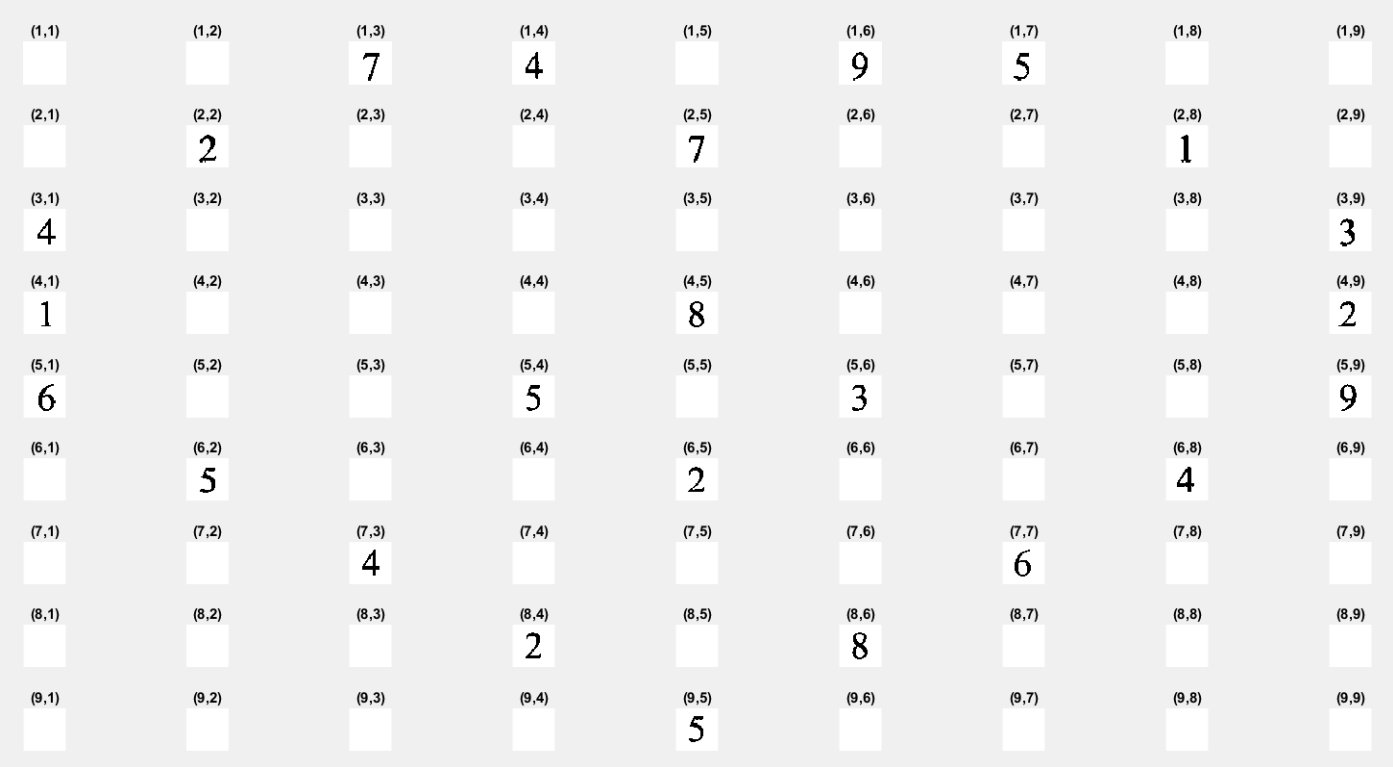
Then, we have resized the image to make it square so that splitting the image will be more convenient.

bw5 = imresize(bw5, [450, 450]);

Besides, we also convert the binary image 'bw5' into a grayscale image, by multiply 255 elementwise.
bw5=double(bw5).*255;
Then, we split the ROI into 81 parts to detect the digits. And, we also remove the 10 pixels from the boundary side for more safety. For this, we first declare a variable named 'subImages' containing 9x9 (total 81) cells. Then, we have stored the splitted 81 images into each cell of 'subImages'.

```
rows = 9;
cols = 9;
subImageHeight = 450 / rows;
subImageWidth = 450 / cols;
subImages = cell(rows, cols);
for i = 1:rows
   for j = 1:cols
      xMin = (j - 1) * subImageWidth + 1;
      yMin = (i - 1) * subImageHeight + 1;
      xMax = j * subImageWidth;
      yMax = i * subImageHeight;
      subImages{i, j} = bw5(yMin+5:yMax-5, xMin+5:xMax-5, :);
   end
end
```

After plotting the 'subImages', it looks like this:



Then, we use OCR to detect digit from each sub-image. If there is no digit detected, we return 0. After that, we store the digits in a matrix.

```
recognizedDigits = cell(size(subImages));

% Processing each sub-image with OCR
for i = 1:rows
    for j = 1:cols
        % Performing OCR on the current sub-image
        results = ocr(subImages{i, j}, 'TextLayout', 'Word', 'CharacterSet', '0123456789');

        % Checking if any digits are recognized
        if ~isempty(results.Words)
            recognizedDigits{i, j} = str2double(results.Words{1});
        else
            recognizedDigits{i, j} = 0;
        end
    end
end

rec_dig_mat=cell2mat(recognizedDigits);
disp(rec_dig_mat);
```

0	0	7	4	0	9	5	0	0
0	2	0	0	7	0	0	1	0
4	0	0	0	0	0	0	0	3
1	0	0	0	8	0	0	0	2
6	0	0	5	0	3	0	0	9
0	5	0	0	2	0	0	4	0
0	0	4	0	0	0	6	0	0
0	0	0	2	0	8	0	0	0
0	0	0	0	5	0	0	0	0

Then, we transfer the matrix into the ‘solveSudoku’ function to get the solved sudoku in a matrix form.

solvedSudoku = solveSudoku(rec_dig_mat);

Sudoku Solving Algorithm (Backtracking method):

Theoretical procedures:

One well-liked strategy for resolving Sudoku puzzles is backtracking. It's a brute-force method that iteratively looks through every alternative in an attempt to find a solution, going backwards when it finds a contradiction. This is how it operates:

1. Select an Empty Cell: In the Sudoku grid, begin with an empty cell.

2. Try a Digit: Attempt to fill the blank cell with a digit (ranging from 1 to 9).

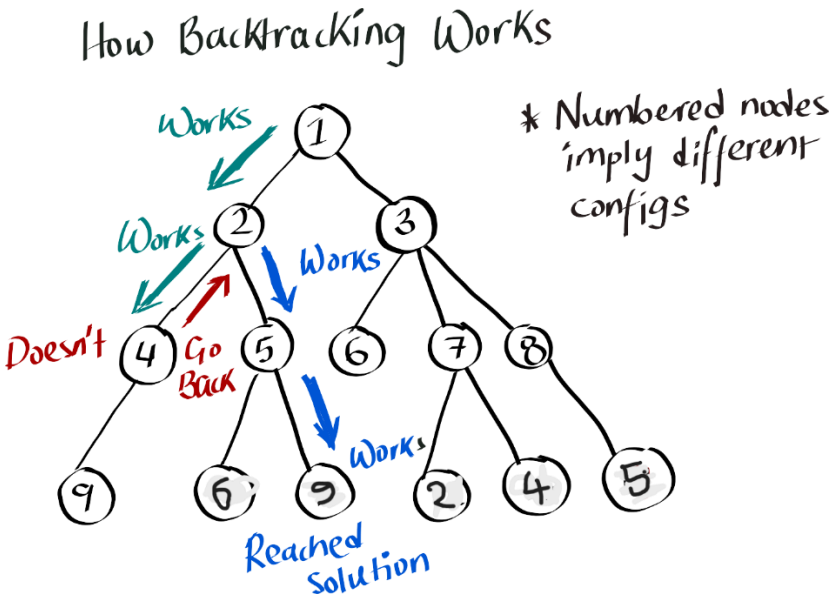
3. Verify Validity: Determine whether the placement of the digits complies with the Sudoku rules.
   - The current row should not include the digit already.
   - The number shouldn't be in the current column already.
   - The current 3x3 subgrid shouldn't already include the digit.

4. Repeat: Go to the next empty cell and repeat steps 2-3 if the digit placement is correct.

5. Backtrack: Go back to the previous cell and try a different digit if there are no valid digits for the current cell. The term "backtracking" originates from this.

6. Repeat Again: Repeat steps 2-5 until a solution is found or until all possible combinations are exhausted.
7. Terminate: Once a solution is found, terminate the process.

Here is a visual representation of backtracking method:



Reference: https://algodaily.com

Implementation:

```
function solvedMatrix = solveSudoku(matrix)
    % Finding an empty cell
    [row, col] = find(matrix == 0, 1);

    % If no empty cell is found, the puzzle is solved
    if isempty(row) || isempty(col)
        solvedMatrix = matrix;
        return;
    end
```

Finding an Empty Cell:

The function first finds an empty cell in the Sudoku grid (represented by a 0 in the matrix). If no empty cell is found, it means the puzzle is already solved, and the function returns the input matrix as the solution.

```
    % Trying placing a digit (1 to 9) in the empty cell
    for digit = 1:9
        % Check if the placement is valid
        if isValid(matrix, row, col, digit)
            % Place the digit in the cell
            matrix(row, col) = digit;

            % Recursively solve the rest of the puzzle
            solvedMatrix = solveSudoku(matrix);

            % If the puzzle is solved, return the solution
            if ~isempty(solvedMatrix)
                return;
            end

            % If placing the current digit doesn't lead to a solution, backtrack
            matrix(row, col) = 0;
        end
    end

    % If no valid digit is found, backtrack
    solvedMatrix = [];
end
```

- *PLACING A DIGIT:*

IT ITERATES OVER DIGITS FROM 1 TO 9 AND TRIES PLACING EACH DIGIT IN THE EMPTY CELL.

- *CHECK VALIDITY:*

FOR EACH DIGIT, IT CHECKS IF PLACING THAT DIGIT IN THE CURRENT CELL IS VALID ACCORDING TO SUDOKU RULES (I.E., THE DIGIT SHOULD NOT ALREADY EXIST IN THE SAME ROW, COLUMN, OR 3X3 SUBGRID).

- *RECURSIVELY SOLVE:*

IF PLACING THE DIGIT IS VALID, IT UPDATES THE MATRIX WITH THE DIGIT AND RECURSIVELY CALLS THE SOLVESUDOKU FUNCTION TO SOLVE THE REST OF THE PUZZLE.

- *BACKTRACK:*

IF THE PUZZLE CANNOT BE SOLVED WITH THE CURRENT DIGIT PLACEMENT, IT BACKTRACKS BY SETTING THE CURRENT CELL BACK TO 0 (EMPTY) AND TRIES THE NEXT DIGIT.

- *RETURN SOLUTION:*

IF A SOLUTION IS FOUND (I.E., THE RECURSIVE CALL RETURNS A NON-EMPTY MATRIX), IT RETURNS THE SOLVED MATRIX.

- *NO VALID SOLUTION:*

IF NO VALID DIGIT CAN BE PLACED IN THE CURRENT CELL, IT RETURNS AN EMPTY MATRIX, INDICATING THAT THE CURRENT PATH DOES NOT LEAD TO A SOLUTION.

Digit validity checking function:

The function code:

```
function isValid = isValid(matrix, row, col, digit)
    % Check if the digit is not in the same row or column
    if any(matrix(row, :) == digit) || any(matrix(:, col) == digit)
        isValid = false;
        return;
    end

    % Check if the digit is not in the same 3x3 subgrid
    subgridRowStart = 3 * floor((row - 1) / 3) + 1;
    subgridColStart = 3 * floor((col - 1) / 3) + 1;

    subgrid = matrix(subgridRowStart:subgridRowStart + 2, subgridColStart:subgridColStart + 2);
    if any(subgrid(:) == digit)
        isValid = false;
        return;
    end

    % If all checks pass, the placement is valid
    isValid = true;
end
```

Explanation:

- Checking Row and Column:

It first checks if the digit already exists in the same row or column by using the function to search for occurrences of the digit in the specified row (matrix(row, :)) or column (matrix(:, col)). If the digit is found in either the row or column, it returns false, indicating that the placement is not valid.

- Checking 3x3 Subgrid:

It then determines the indices of the 3x3 subgrid containing the cell by calculating the starting row (subgridRowStart) and column (subgridColStart) indices based on the cell's row and column indices.

It extracts the subgrid from the Sudoku puzzle grid using the calculated indices.

It checks if the digit already exists in the extracted subgrid using the any function. If the digit is found in the subgrid, it returns false, indicating that the placement is not valid.

- Final Validation:

If the digit is not found in the same row, column, or 3x3 subgrid, the function returns true, indicating that the placement is valid.

Converting the solved sudoku into image:

To convert the solved sudoku into matrix, we need to follow certain steps:

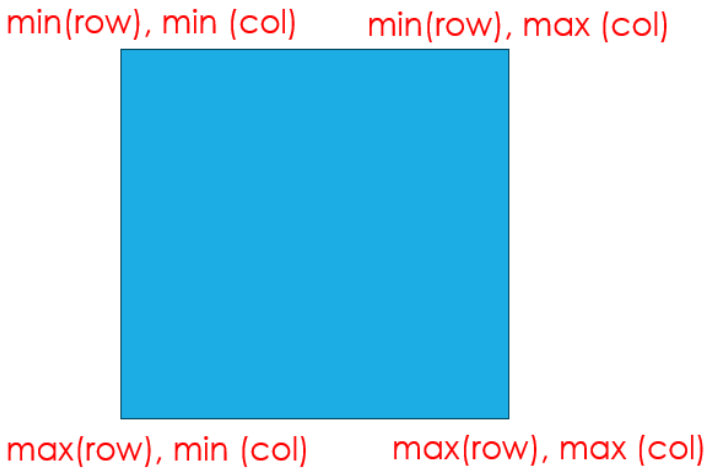
- Cropping the sudoku image from the original image.
- Extract the digit from the solved sudoku.
- Compute the position (original image) in pixels based on the row and column.
- Overlaying the digit on the original image.

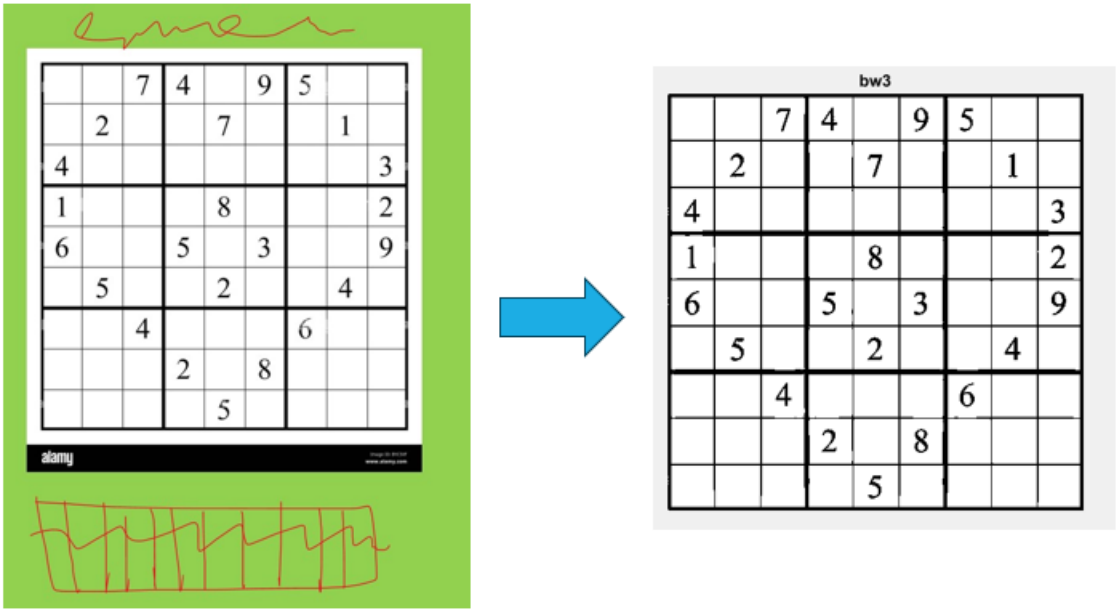
Cropping the sudoku image from the original image:

To crop the sudoku image we need to find the corner point of the sudoku. Then we can easily crop the image.

```
% Assuming 'originalImage' is your original image
overlayedImage = sudoku(min(row):max(row),min(col):max(col));
```

Through the upper code we can easily mark the corner point. Then we can crop it.





Here we successfully crop the sudoku.

Extract the digit from the solved sudoku:

Here we need to read the solved sudoku matrix and store the solved digit in a variable.

```
% Reading the solution on the original image
for i = 1:size(solvedSudoku, 1)
    for j = 1:size(solvedSudoku, 2)
        digit = solvedSudoku(i, j);
```

By the code we can extract the digit.

Compute the position (original image) in pixels based on the row and column;

Now we need to find the position where the solved digits need to put on. We will do this by pixel calculation based on row and column.

```
% Compute the position in pixels based on the row and column
x = (j - 1) * subImageWidth + 1 + offset
y = (i - 1) * subImageHeight + 1 + offset;
```

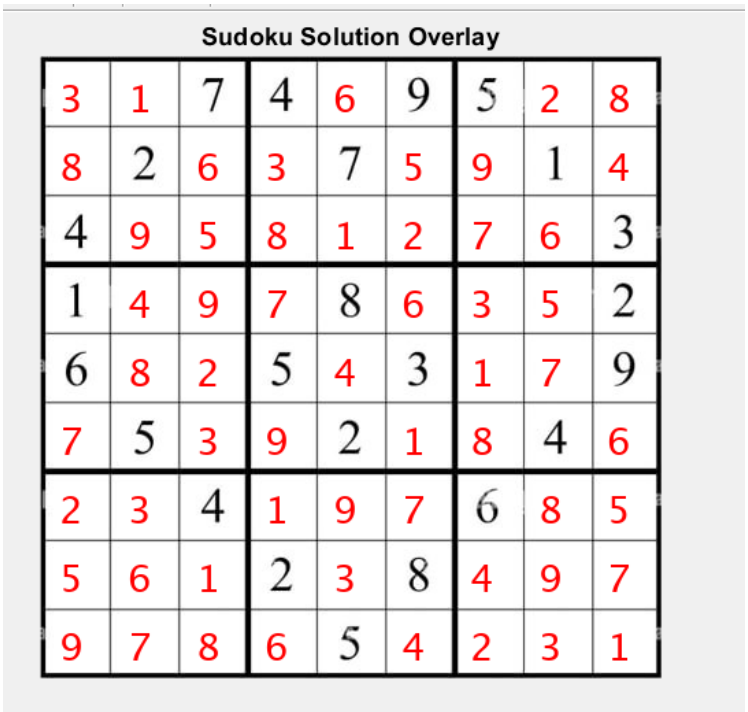
Overlaying the digit on the original image:

At the very begin of image processing, we mark the given digit in the sudoku as 1 and the digit we need to find as 0. To overlay the digit in the image, we will use the same argument.

We will check if there is any 0 mark, we will overlay the position by the solved digit.

```
if rec_dig_mat(i,j)==0
% Overlaying the digit on the original image
    overlayedImage = insertText(overlayedImage, [x, y], num2str(digit), 'FontSize', 28, 'BoxColor', ...
        'white', 'TextColor', 'red', 'BoxOpacity', 0);
end
```

0	1	1
0	1	0
1	0	1



By following these steps we can easily convert the solved sudoku into image.

Overall code for converting into image:

```
% Assuming 'originalImage' is your original image
overlayedImage = sudoku(min(row):max(row),min(col):max(col));

% Assuming 'subImageHeight' and 'subImageWidth' are calculated earlier
offset = 5;

% Reading the solution on the original image
for i = 1:size(solvedSudoku, 1)
    for j = 1:size(solvedSudoku, 2)
        digit = solvedSudoku(i, j);

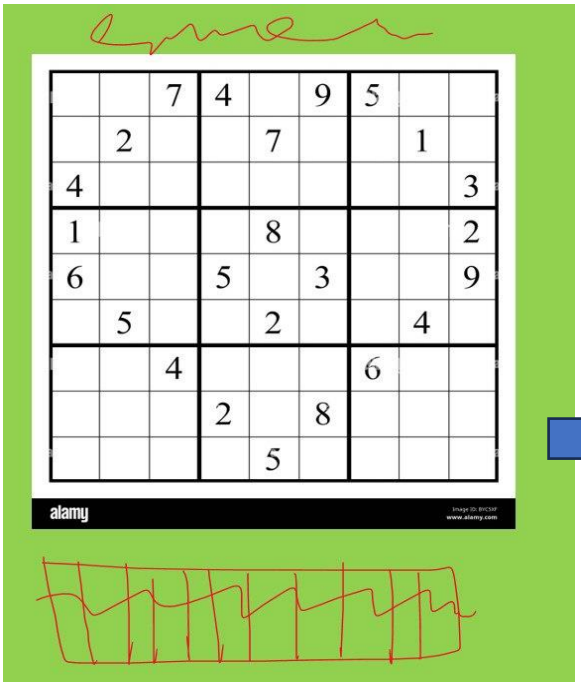
        % Compute the position in pixels based on the row and column
        x = (j - 1) * subImageWidth + 1 + offset
        y = (i - 1) * subImageHeight + 1 + offset;

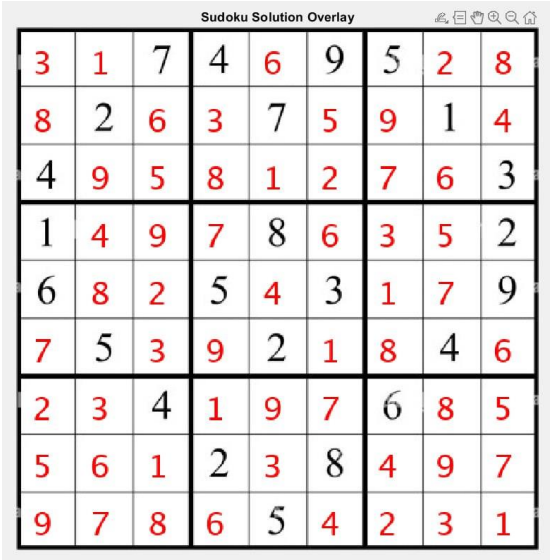
        if rec_dig_mat(i,j)==0
        % Overlaying the digit on the original image
            overlayedImage = insertText(overlayedImage, [x, y], num2str(digit), 'FontSize', 28, 'BoxColor', ...
                'white', 'TextColor', 'red', 'BoxOpacity', 0);
        end
    end
end

% Display the overlayed image
figure;
imshow(overlayedImage);
title('Sudoku Solution Overlay');
impixelinfo;
```

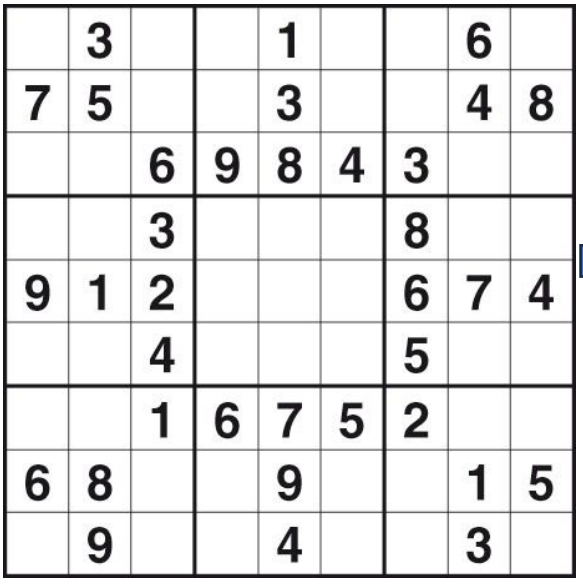
Sample input-output:

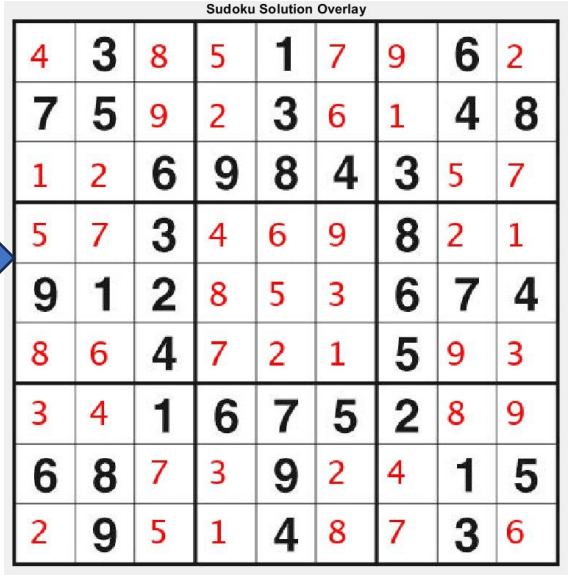
Sample1:





Sample2:





Limitation:

We have certain limitations in our project. Our project cannot detect hand writing. It cannot identify image if the image is blur or it is making an angle.

Future Aspect:

We want work with our limitations. We want work on our project so that it can detect hand writing, blur images, and bent images.

Discussion:

The development of an automated Sudoku solver utilizing image processing techniques represents a significant advancement in both computer vision and puzzle-solving domains. By harnessing the power of image recognition algorithms, this system can analyze and interpret the grid structure and numerical content of a Sudoku puzzle from a given image.

At its core, the solver employs a series of steps to extract and process the Sudoku grid from the input image. Initially, the image undergoes preprocessing techniques such as thresholding, blurring, and edge detection to enhance the grid's visibility and isolate relevant features. Subsequently, contour detection algorithms are utilized to identify the boundaries of the Sudoku grid. Through approximation and analysis of these contours, the largest rectangle representing the Sudoku grid is determined.

Once the grid is accurately extracted, the system proceeds to digit recognition, where individual cells within the grid are analyzed to identify the numerical values present. This step often involves machine learning models trained on digit datasets to accurately classify and extract the digits.

Finally, with the grid structure and numerical content extracted, the solver employs efficient Sudoku solving algorithms such as backtracking or constraint satisfaction to find the solution. The solution is then overlaid onto the original image or presented in a digital format.

The implications of such a system are vast, ranging from entertainment and educational purposes to practical applications in automation and robotics. By leveraging image processing and machine learning, the automated Sudoku solver showcases the potential of artificial intelligence to tackle complex problem-solving tasks in a variety of domains, paving the way for further advancements in intelligent systems and automation technologies.

Contributions:

Name & IDs:	*Photo*	*Contributions:*
1. Moshiur Rahman - 2006099		*Sudoku Puzzle Extraction from an Image using Image processing and convert it to a 9*9 Matrix*
2. Shahriar Rashid Khan Sifat -2006117		*Sudoku Solving Algorithm Development and Implementation in MATLAB*
3. Tanvir Arafat Fahim -2006118		*Converting the Solved Sudoku into Output Image, limitations, future aspect, final report editing.*
4. Md. Sazidur Rahman - 2006121		*Image Processing Algorithm Implementation in MATLAB & Overall Overview on Sudoku Solving Method.*